22I-2327 Mohammad Rohaan Assignment# 03

Problem 1 – Adaptive String-Matching Strategy

Idea:

We automatically choose between Naive, KMP, and Rabin–Karp based on pattern length, alphabet size and expected collisions. For very small inputs we use Naive, for small alphabets / repetitive texts we use KMP, and in normal cases we start with Rabin–Karp and switch to KMP if we see too many hash collisions.

Decision Rules:

- If m ≤ 3 or n*m ≤ 20,000 → use Naive.

- Else if alphabet size ≤ 4 → use KMP.

- Else if number of patterns k > 1 and m ≥ 4 → use Rabin–Karp style (conceptually multi-pattern).

- Otherwise → start with Rabin–Karp, and fall back to KMP if collisions are high.

Pseudocode:

```
function ADAPTIVE_MATCH(text T, pattern P, numPatterns k):
    n = length(T), m = length(P)
    if m == 0 or n == 0:
        return empty list
```

```
    work = n * m
    if m <= 3 or work <= 20000:
        return NAIVE_MATCH(T, P)
    alphabetSize = ESTIMATE_ALPHABET(T)
    if alphabetSize <= 4:
        return KMP_MATCH(T, P)

    if k > 1 and m >= 4:
        return RABIN_KARP_WITH_SWITCH(T, P)
    return RABIN_KARP_WITH_SWITCH(T, P)
```

In RABIN_KARP_WITH_SWITCH we count collisions (hash matches but characters differ).
If collisionCount exceeds a threshold (like max(5, 2*sqrt(n))) we stop Rabin–Karp
and instead run KMP on the same text and pattern.

Time and Space:

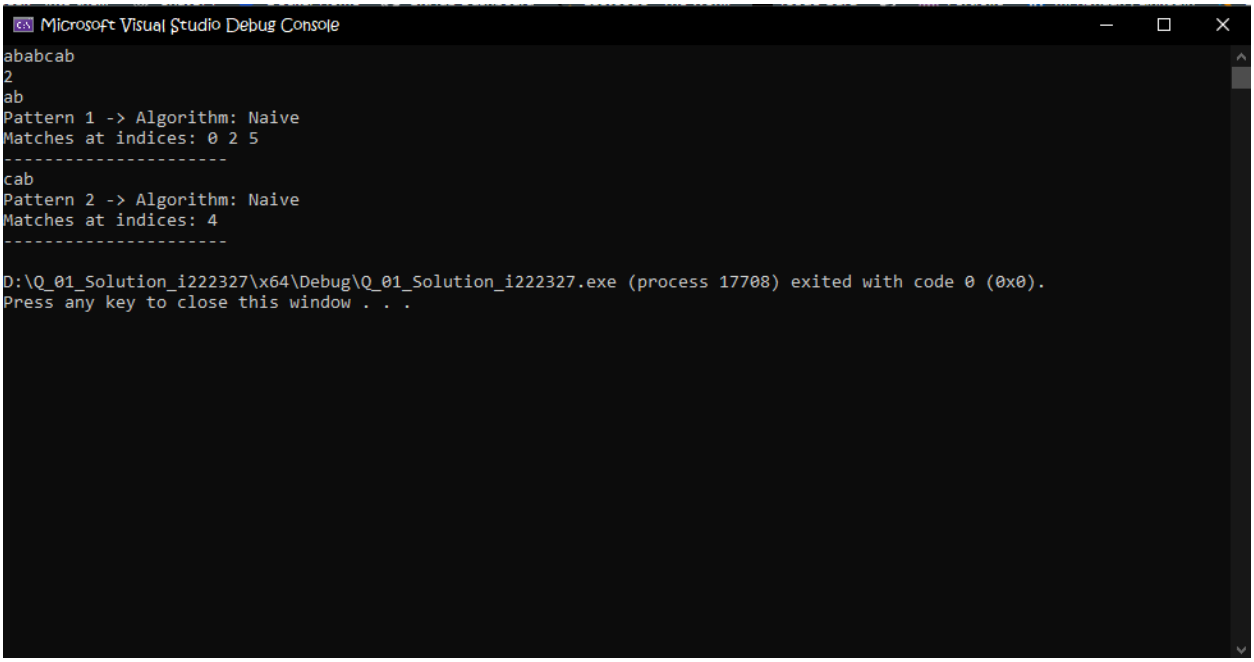
- Naive: Time O(n*m), Space O(1)
- KMP: Time O(n + m), Space O(m)
- Rabin–Karp: Avg Time O(n + m), worst O(n*m), Space O(1)
- Hybrid (RK with fallback to KMP): Avg ~ O(n), worst O(n + m)

Example:

Text = "aaaaaaabaaaaaa"

Pattern = "aaaaab"

Alphabet is small and text is repetitive, so KMP is chosen. If we started with RK,
many collisions would occur and the algorithm would switch to KMP.



Problem 2 – Divide and Conquer Text Similarity

Idea:

We compute the maximum common substring length between A and B using binary search on the length. For a candidate length L, we use rolling hash to check if any substring of length L in A also appears in B.

Pseudocode:

function HAS_COMMON_OF_LENGTH(A, B, L):

    if L == 0: return true

    if L > |A| or L > |B|: return false

    hash all substrings of length L in A into a set S

```
    hash all substrings of length L in B and check membership in S
    return true if any match, else false

function MAX_COMMON_LENGTH(A, B):
    low = 0
    high = min(|A|, |B|)
    best = 0
    while low <= high:
        mid = (low + high) / 2
        if HAS_COMMON_OF_LENGTH(A, B, mid):
            best = mid
            low = mid + 1
        else:
            high = mid - 1
    return best
```

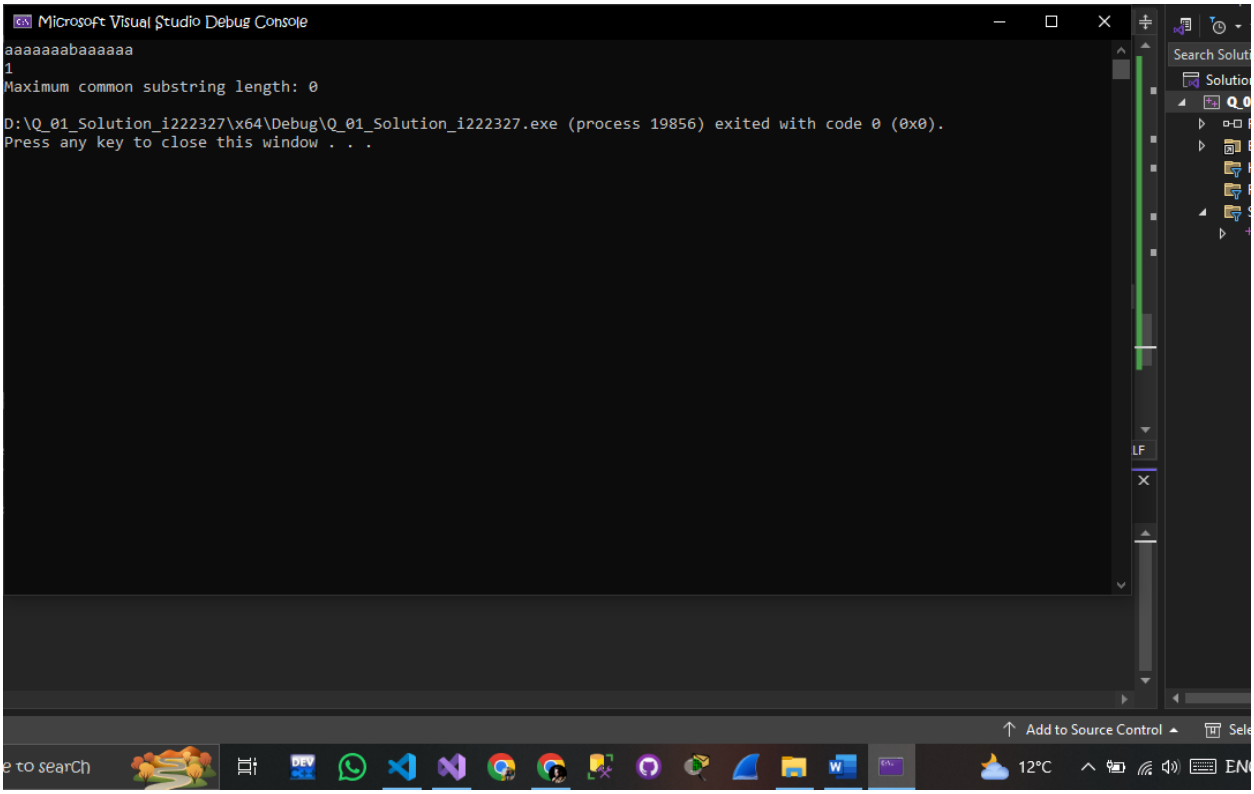
Time and Space:

- Each HAS_COMMON_OF_LENGTH: O(n + m)
- Binary search: O(log(min(n, m))) steps
- Total: O((n + m) * log(min(n, m))) time, O(n) space.

Example:

For A = "abcxabcdabxabcdabcdabcy" and B = "xabcabcdabcy"

the algorithm finds the length of the longest common substring using this binary search over lengths.



Problem 3 – Optimal Multi-pattern Matching Automaton

Idea:

We use Aho–Corasick as a base multi-pattern matcher which naturally supports overlapping matches. We make the matching case-insensitive by converting characters to lowercase in the automaton. Wildcards (?) and memory optimizations are mainly discussed conceptually in the report, while the code implements the standard core.

Memory-Optimized Concepts:

- Use only existing edges (sparse representation instead of full alphabet matrix).
- Group patterns and build smaller automatons (divide and conquer) to reduce peak memory.
- For wildcards ?, treat them as transitions that can match any symbol, or expand them at build time where needed.

Pseudocode for building (simple AC):

```
BUILD_TRIE(patterns):
    create root
    for each pattern p:
        walk/create nodes for its characters
        mark end node with pattern id

BUILD_FAILURE_LINKS():
    use BFS from root
    for each node, compute fail link and merge outputs from fail link

MATCH(text):
    walk through text, following edges and fail links
```

whenever we reach a node with outputs, report matches

Time and Space:

- Build: O(total length of patterns)
- Match: O(length of text + number of matches)
- Space: O(number of states), roughly O(total pattern length)

Example:

Patterns: "he", "she", "his", "hers"

Text: "ahishers"

The automaton reports several overlapping matches, for example "he" inside "she" and "hers", and "his" starting at index 1.

Problem 4 – Divide and Conquer in String Compression

Idea:

We approximate the core of substring-based compression by finding the longest repeated substring in a string. We again use binary search on the length L and a rolling hash to test if there is a repeated substring of that length.

Pseudocode:

```
function HAS_REPEATED_OF_LENGTH(S, L):
    if L == 0: return true
    if L > |S|: return false
    compute all hashes of substrings of length L using rolling hash
    store them in a set
    if any hash appears twice, return true, else false

function LONGEST_REPEATED_SUBSTRING_LENGTH(S):
    low = 0
    high = |S|
    best = 0
    while low <= high:
        mid = (low + high) / 2
        if HAS_REPEATED_OF_LENGTH(S, mid):
            best = mid
            low = mid + 1
        else:
            high = mid - 1
    return best
```

Time and Space:

- Each HAS_REPEATED_OF_LENGTH: O(n)

- Binary search over L: O(log n)

- Total: O(n * log n) time, O(n) space.

Example:

For S = "banana", the longest repeated substring is "ana" with length 3.

Relation to Compression:

Real compressors like LZ77 and BWT rely on repeated substrings and suffix structures.

The binary-search + hash approach is a simpler algorithmic version. Suffix arrays or suffix trees can find the longest repeated substring in O(n log n) or O(n), but are more complex to implement.

